

**BỘ GIÁO DỤC & ĐÀO TẠO
ĐẠI HỌC KINH TẾ TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN KINH DOANH**



**BÁO CÁO ĐỒ ÁN CUỐI KỲ HỌC PHẦN
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT**

ĐỀ TÀI:

**THUẬT TOÁN CHU-LIU/EDMONDS VÀ CẤU TRÚC DỮ LIỆU
CHO BÀI TOÁN CÂY KHUNG NHỎ NHẤT CÓ HƯỚNG**

GIẢNG VIÊN HƯỚNG DẪN: TS. Huỳnh Văn Đức

MÃ HỌC PHẦN: 26D3INF50901101

NHÓM SINH VIÊN THỰC HIỆN:	Dương Minh Triết	35254020185
	Nguyễn Thị Kỳ Duyên	31231026925
	Trương Thị Kiều An	35254020035
	Lại Thị Hiền	35254020060
	Dương Minh Tú	35254020119

TP. Hồ Chí Minh, ngày 11 tháng 6 năm 2026

MỤC LỤC

MỤC LỤC.....	2
PHÂN CÔNG CÔNG VIỆC THÀNH VIÊN	3
LỜI CẢM ƠN.....	4
TÓM TẮT.....	5
MỞ ĐẦU	6
1. Bài toán ứng dụng	6
2. Bài toán mà thuật toán sẽ giải.....	6
3. Ý tưởng thuật toán	6
4. Ví dụ minh họa	7
5. Mục tiêu bài luận.....	9
CƠ SỞ LÝ THUYẾT.....	10
1. Thuật toán Chu-liu/Edmonds	10
2. Các cấu trúc dữ liệu liên quan	11
ĐỀ XUẤT CẤU TRÚC DỮ LIỆU.....	15
1. Yêu cầu về cấu trúc dữ liệu	15
2. Đề xuất các cấu trúc dữ liệu	15
3. Tổng hợp lựa chọn cấu trúc dữ liệu và thuật toán.....	20
THỰC HIỆN	22
1. Chạy trên ví dụ đơn giản	22
2. Chạy trên ví dụ phức tạp.....	23
3. Kết luận	25
4. Mã nguồn và kết quả.....	26
REFERENCES.....	28

PHÂN CÔNG CÔNG VIỆC THÀNH VIÊN

STT	Họ và tên	MSSV	Nhiệm vụ được giao	Mức độ hoàn thành (%)
1	Dương Minh Triết (NT)	35254020185	- Cài đặt thuật toán chính Chu - liu/Edmonds - Viết phần Thực hiện chạy demo, tóm tắt, mở đầu, kết luận - Viết Chọn cấu trúc dữ liệu - Phần 2.4 - Nghiên cứu lý thuyết, chỉnh sửa bài viết, cải tiến code của thành viên - Kiểm tra đạo văn, chỉnh sửa định dạng	100%
2	Nguyễn Thị Kỳ Duyên	31231026925	- Cài đặt class UnionFind - Viết Chọn cấu trúc dữ liệu - Phần 1, 2.1, 2.2, 3 - Làm file thuyết trình	100%
3	Trương Thị Kiều An	35254020035	- Cài đặt class Arborescence - Viết Cơ sở lý thuyết - Phần 2	100%
4	Lại Thị Hiền	35254020060	- Cài đặt class Edge - Viết Chọn cấu trúc dữ liệu - Phần 2.3	100%
5	Dương Minh Tú	35254020119	- Cài đặt class DirectedGraph - Viết Cơ sở lý thuyết - Phần 1	100%

LỜI CẢM ƠN

Để hoàn thành đề tài "Thuật toán Chu-Liu/Edmonds và cấu trúc dữ liệu cho bài toán cây khung nhỏ nhất có hướng", nhóm chúng em xin gửi lời cảm ơn chân thành đến Thầy Huỳnh Văn Đức - giảng viên học phần Cấu trúc dữ liệu và Giải thuật, Khoa Công nghệ Thông tin Kinh doanh, Trường Đại học Kinh tế TP. Hồ Chí Minh đã tận tình giảng dạy, truyền đạt những kiến thức nền tảng quý báu về cấu trúc dữ liệu và giải thuật trong suốt thời gian học phần, tạo điều kiện để nhóm có đủ kiến thức và công cụ cần thiết thực hiện đề tài này.

Trong quá trình tìm hiểu và triển khai đề tài, do kiến thức về thuật toán Chu-Liu/Edmonds còn khá mới và phức tạp, nhóm không tránh khỏi những khó khăn và thiếu sót nhất định trong quá trình nghiên cứu lý thuyết, thiết kế cấu trúc dữ liệu cũng như cài đặt và thực nghiệm chương trình. Nhóm rất mong nhận được sự góp ý, nhận xét từ quý thầy/cô để bài báo cáo được hoàn thiện hơn.

Cuối cùng, nhóm xin chân thành cảm ơn các thành viên đã cùng nhau nỗ lực, phối hợp chặt chẽ trong suốt quá trình nghiên cứu, thảo luận và hoàn thiện đề tài này.

Nhóm xin chân thành cảm ơn!

TÓM TẮT

Bài luận trình bày về thuật toán Chu-Liu/Edmonds - một thuật toán thời gian đa thức dùng để tìm Cây khung nhỏ nhất có hướng (Minimum Spanning Arborescence - MSA) trên đồ thị có hướng có trọng số, với ứng dụng thực tiễn trong các lĩnh vực như xử lý ngôn ngữ tự nhiên (Dependency Parsing), mạng viễn thông (Multicast Routing), hệ thống phân tán (Replication Topology) và logistics.

Nội dung bài luận xoay quanh ý tưởng cốt lõi của thuật toán: tiếp cận tham lam bằng cách chọn cạnh đi vào có trọng số nhỏ nhất cho mỗi đỉnh, sau đó phát hiện và xử lý các chu trình bằng cách co (contract) chúng thành siêu nút, lặp lại quá trình đệ quy cho đến khi không còn chu trình, cuối cùng thực hiện pha bung (expand) để truy vết và xây dựng kết quả cuối cùng.

Trên cơ sở phân tích đặc điểm hoạt động của thuật toán (đệ quy co/bung đồ thị, biến đổi liên tục cấu trúc dữ liệu qua nhiều tầng), nhóm đề xuất bốn cấu trúc dữ liệu chính để cài đặt: Edge (cạnh, có cơ chế lưu vết `real_edge` để truy vết qua các tầng co), `DirectedGraph` (đồ thị biểu diễn dạng Edge List để tối ưu bộ nhớ), Union-Find (Disjoint Set Union để gom nhóm siêu nút với độ phức tạp gần hằng số), và Arborescence (cấu trúc lưu kết quả cuối cùng kèm cơ chế tự kiểm tra tính hợp lệ).

Phần thực nghiệm cài đặt chương trình bằng Python, kiểm thử trên các bộ dữ liệu từ quy mô nhỏ đến lớn (5,000 đến 50,000 cạnh). Kết quả cho thấy thuật toán đạt độ chính xác 100% (Valid: True) ở mọi quy mô, mức tiêu thụ bộ nhớ trung bình thấp (tối đa 88 MB ở 50,000 cạnh), tuy nhiên thời gian thực thi trung bình tăng đáng kể ở quy mô lớn (từ dưới 2 giây ở 10,000 cạnh lên đến gần 47 giây ở 50,000 cạnh) do số lượng lệnh gọi đệ quy co/bung tăng theo cấp số nhân.

MỞ ĐẦU

1. Bài toán ứng dụng

- Xử lý Ngôn ngữ tự nhiên (NLP) - Dependency Parsing (Phân tích cú pháp phụ thuộc): Cho một câu nói, tìm cách để máy tính hiểu được cấu trúc ngữ pháp (từ nào bổ nghĩa cho từ nào, đâu là chủ ngữ, đâu là vị ngữ).
- Mạng viễn thông & CDN (Multicast Routing / Broadcast): Có một máy chủ trung tâm phát sóng trực tiếp một trận bóng đá. Ta cần truyền luồng video này đến hàng triệu máy chủ biên trên toàn cầu.
- Hệ thống Phân tán (Distributed Systems) - Replication Topology: Ghi một dòng dữ liệu vào Master database ở Region US-East. Dữ liệu này phải được đồng bộ sang 10 database khác ở Châu Âu, Châu Á,...
- Logistics & Tối ưu hóa chuỗi cung ứng (Supply Chain): Thiết kế mạng lưới luân chuyển hàng hóa từ Kho tổng (Hub) về các bưu cục nhỏ.

2. Bài toán mà thuật toán sẽ giải

- Mô tả bài toán: Cho một đồ thị có hướng $G = (V, E)$ bao gồm V đỉnh (được đánh số từ 0 đến $V - 1$) và E cạnh có hướng. Mỗi cạnh nối từ đỉnh u đến đỉnh v mang một trọng số thực w . Cho trước một đỉnh gốc $root \in V$.
- Nhiệm vụ bài toán: Tìm một tập hợp các cạnh là một phần của E sao cho tạo thành một Cây khung có hướng nhỏ nhất (Arborescence) thỏa mãn các điều kiện sau:
 - + Đỉnh root không có cạnh nào trở vào (In-degree = 0).
 - + Tất cả các đỉnh còn lại có đúng 1 cạnh trở vào (In-degree = 1).
 - + Từ đỉnh root có thể đi đến được tất cả các đỉnh khác trong đồ thị.
 - + Tổng trọng số của các tập hợp cạnh được chọn là nhỏ nhất có thể.

3. Ý tưởng thuật toán

- Mục tiêu đầu tiên là tham lam nhất có thể. Có nghĩa là phải tìm đường đi có trọng số ít nhất, tại mỗi nút thì nhìn xem nút nào nối vào mình có trọng số nhỏ nhất là chọn.

Đối với trường hợp lý tưởng, tất cả các nút đều được nối thông suốt từ root - bài toán hoàn thành. Nhưng thực tế rằng đồ thị có hướng rất dễ dẫn đến trường hợp có chu trình (Ví dụ: Nút A thấy trọng số từ A đến B thấp nhất, nút B cũng nhận thấy như vậy ngược lại. Từ đó hình thành một vòng tròn khép kín và bị cô lập khỏi root).

- Từ đó, nhóm nhận thấy phải tìm cách sửa lỗi sai. Cần phải xử lý việc làm sao để không dính chu trình. Thay vì bỏ đường đó đi và tìm lại từ đầu, ta có thể bọc nó lại thành một siêu nút (super-node). Trong siêu nút này sẽ chứa các nút và trọng số đã được nối với nhau. Sau đó,

cần phải tìm các cạnh bên ngoài (có thể từ gốc) để nối vào siêu nút này. Các cạnh bên ngoài sẽ so sánh chi phí cơ hội với nhau và chọn cạnh có chi phí cơ hội nhỏ nhất, sau đó phá cạnh ở bên trong của nút đó.

- Lặp lại quá trình tham lam và gộp vòng trên đồ thị mới (đã bị co nhỏ lại) cho đến khi không còn chu trình nào nữa, từ đây có thể chắc chắn rằng có một con đường từ gốc đến tất cả các nút. Cuối cùng là cần phải đi kiểm tra ngược lại từ trong ra ngoài, kiểm tra xem cạnh đi ra ngoài nối vào nút nào, kiểm tra xem nút đó có được trở vào hay không, nếu có thì ngay lập tức bỏ ra.

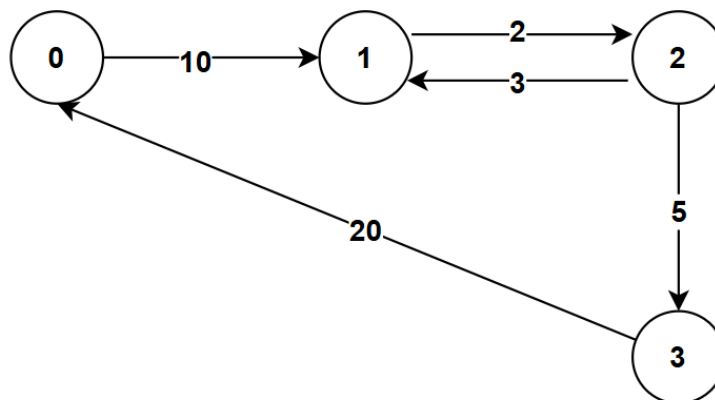
4. Ví dụ minh họa

Dữ liệu input:

- Bao gồm 4 nút (đã bao gồm root) và 5 cạnh. Nút sẽ theo format **V (Số nút) E (Số cạnh) root (optional)**. Và cạnh sẽ theo format **E lines: u (Cạnh 1) v (Cạnh 2) w (Trọng số)**.

```
1. 4 5 0
2. 0 1 10
3. 1 2 2
4. 2 1 3
5. 2 3 5
6. 0 3 20
```

Sơ đồ đồ thị ban đầu:



Hình 1. Đồ thị ban đầu theo input

Giải theo từng bước:

- **Bước 1:** Tìm cạnh vào nhỏ nhất. Thuật toán đứng tại các đỉnh (trừ Root 0) và chọn cạnh có trọng số nhỏ nhất trở vào mình:

+ Đỉnh 1: Có hai cạnh trở vào là $0 \rightarrow 1$ ($w=10$) và $2 \rightarrow 1$ ($w=3$). Trọng số nhỏ nhất là: $2 \rightarrow 1$ ($W_{in} = 3$).

+ Đỉnh 2: Chỉ có một cạnh trở vào là $1 \rightarrow 2$ ($W_{in} = 2$).

+ Đỉnh 3: Có hai cạnh trở vào là $2 \rightarrow 3$ ($w=5$) và $0 \rightarrow 3$ ($w=20$). Trọng số thấp nhất là: $2 \rightarrow 3$ ($W_{in} = 5$).

Suy ra, tập cạnh tạm chọn: $[2 \rightarrow 1, 1 \rightarrow 2, 2 \rightarrow 3]$

Bước 2: Phát hiện chu trình

- Thuật toán kiểm tra tập cạnh tạm chọn và phát hiện chu trình: $1 \rightarrow 2 \rightarrow 1$.

- Đỉnh 3 nhận cạnh từ 2, nhưng cả cụm $1 \leftrightarrow 2$ này đang bị cô lập, không có đường đi từ Root 0 đến.

Bước 3: Co chu trình (Tạo siêu nút)

- Gộp đỉnh 1 và 2 thành siêu nút đặt tên là **S**. Bây giờ, đồ thị mới (Contracted Graph) sẽ thu gọn lại chỉ còn 3 đỉnh: 0, S, 3.

- Thuật toán tiền hành tính lại giá trị W_{new} cho các cạnh từ ngoài đâm vào siêu nút S:

Cạnh $0 \rightarrow 1$ ($w=10$) nối vào đỉnh 1 (đỉnh 1 có $W_{in} = 3$):

$$W_{new} = 10 - 3 = 7$$

Cạnh này biến thành: $0 \rightarrow S$ ($w=7$)

Cạnh $2 \rightarrow 3$ và $0 \rightarrow 3$ giữ nguyên vì đỉnh đích là 3 (nằm ngoài chu trình).

Bước 4: Độ quy trên đồ thị co

- Đồ thị lúc này chỉ còn: 0, S, 3. Tiếp tục tham lam:

Nút S: Cạnh rẻ nhất trở vào nó là $0 \rightarrow S$ ($W_{new} = 7$).

Nút 3: Cạnh rẻ nhất trở vào nó là $2 \rightarrow 3$ ($w = 5$).

Không còn chu trình. Đồ thị co đã được giải quyết xong.

Bước 5: Bung chu trình. Thuật toán quay ngược trở lại đồ thị gốc để sửa sai:

- Cạnh ngoài được chọn là $0 \rightarrow S$ (bản chất gốc là cạnh $0 \rightarrow 1$).

- Đúng như logic đã phát biểu: Vì cạnh ngoài đâm vào đỉnh **1**, thuật toán sẽ bỏ cạnh nội bộ trở vào đỉnh **1** (xóa bỏ cạnh $2 \rightarrow 1$).
- Cạnh nội bộ còn lại trong chu trình là $1 \rightarrow 2$ được GIỮ LẠI.
- Cạnh kết nối ra ngoài $2 \rightarrow 3$ được GIỮ LẠI.

Output cuối cùng:

```
Hợp lệ: True
Tổng trọng số: 17.0
0 -> 1 : 10.0
1 -> 2 : 2.0
2 -> 3 : 5.0
```

5. Mục tiêu bài luận

Trên cơ sở phân tích bài toán Cây khung nhỏ nhất có hướng (Minimum Spanning Arborescence) và thuật toán Chu-Liu/Edmonds, đề tài hướng đến các mục tiêu cụ thể sau:

- Tìm hiểu và trình bày cơ sở lý thuyết của thuật toán Chu-Liu/Edmonds, bao gồm lịch sử phát triển, nguyên lý hoạt động (cơ chế tham lam, phát hiện chu trình, co/bung đồ thị) và độ phức tạp thời gian của thuật toán.
- Nghiên cứu các cấu trúc dữ liệu nền tảng liên quan (đồ thị có hướng, cây, cây khung có hướng gốc R, Union-Find) làm cơ sở lý luận cho việc thiết kế cấu trúc dữ liệu cài đặt thuật toán.
- Đề xuất và thiết kế các cấu trúc dữ liệu phù hợp (Edge, DirectedGraph, UnionFind, Arborescence) nhằm tối ưu về tốc độ xử lý và bộ nhớ, đáp ứng yêu cầu hoạt động đệ quy co/bung của thuật toán trên các tập dữ liệu lớn.
- Cài đặt chương trình minh họa thuật toán Chu-Liu/Edmonds bằng ngôn ngữ Python dựa trên các cấu trúc dữ liệu đã đề xuất, kèm giao diện dòng lệnh (CLI) để kiểm thử.
- Thực nghiệm và đánh giá hiệu năng của chương trình (tính đúng đắn, thời gian thực thi, mức tiêu thụ bộ nhớ) trên các bộ dữ liệu có quy mô khác nhau (từ 5,000 đến 50,000 cạnh), từ đó rút ra nhận xét về khả năng ứng dụng thực tế của thuật toán.

CƠ SỞ LÝ THUYẾT

1. Thuật toán Chu-liu/Edmonds

1.1. Lịch sử phát triển thuật toán

Thuật toán Edmonds, còn được gọi là thuật toán Chu–Liu/Edmonds, là một thuật toán thời gian đa thức trong lý thuyết đồ thị dùng để tìm cây khung có hướng (spanning arborescence) trọng số nhỏ nhất có gốc tại một đỉnh cho trước trong một đồ thị có hướng với các cạnh được gán trọng số.[1] Thuật toán này được đề xuất độc lập bởi Yoeng-Jin Chu và Tseng-Hong Liu vào năm 1965, sau đó được công bố bởi Jack Edmonds trong bài báo Optimum Branchings xuất bản năm 1967 trên Journal of Research of the National Bureau of Standards.[1] Thuật toán giải quyết bài toán arborescence chi phí tối thiểu (minimum cost arborescence problem), một sự tổng quát hóa của bài toán cây khung nhỏ nhất (minimum spanning tree problem) sang đồ thị có hướng bằng cách tìm một cây có hướng đi từ đỉnh gốc đến tất cả các đỉnh với tổng trọng số cạnh nhỏ nhất.

1.2. Cơ chế hoạt động của thuật toán

Cơ chế hoạt động ở mức khái quát bao gồm việc lặp lại quá trình chọn, đối với mỗi đỉnh không phải đỉnh gốc, cạnh đi vào có trọng số nhỏ nhất để tạo thành một rừng giả (pseudoforest). Nếu phát hiện chu trình, thuật toán sẽ co (contract) mỗi chu trình thành một siêu nút (supernode), điều chỉnh trọng số các cạnh bằng cách trừ đi trọng số cạnh nhỏ nhất trong chu trình nhằm bảo đảm tính tối ưu, rồi đệ quy trên đồ thị đã được rút gọn. Khi không còn chu trình nào, các phép co được mở rộng trở lại và các cạnh được lựa chọn để hình thành arborescence. Quá trình này bảo đảm nghiệm thu được là tối ưu vì việc điều chỉnh trọng số vẫn bảo toàn tổng chi phí của lời giải.[2]

1.3. Độ phức tạp thuật toán (Time Complexity)

Thuật toán blossom của Edmonds để tìm ghép cặp cực đại theo lực lượng (maximum cardinality matching) trong một đồ thị vô hướng có n đỉnh và m cạnh hoạt động theo các pha (phase), trong đó mỗi pha cố gắng tìm một đường tăng cường (augmenting path) đối với ghép cặp hiện tại, đồng thời có thể co các blossom (chu trình lẻ) thành các siêu nút (supernode). Trong trường hợp xấu nhất, có $O(n)$ pha như vậy, vì mỗi pha thành công làm tăng kích thước ghép cặp thêm một đơn vị, và kích thước ghép cặp cực đại không vượt quá $\frac{n}{2}$.

Phiên bản gốc của Edmonds đạt độ phức tạp thời gian $O(n^4)$, chủ yếu do cần $O(n^2)$ thời gian cho mỗi pha để tìm kiếm đường tăng cường bằng các phương pháp gán nhãn (labeling) và co (contraction) đơn giản trên $O(n)$ pha, trong khi mỗi thao tác co và giãn (expansion) blossom tiêu tốn $O(n^2)$ thời gian trong các đồ thị dày đặc.[3]

Các phiên bản cải tiến đã giảm được cận trên này. Phiên bản của Gabow năm 1976 sử dụng các cấu trúc dữ liệu tinh vi, chẳng hạn như cây phân cấp (hierarchical tree) để quản lý

blossom và các biến thể union-find hiệu quả cho các phép co, đạt độ phức tạp tổng thể $O(n^3)$. Kết quả này đạt được bằng cách thực hiện tìm kiếm đường đi với chi phí $O(n^2)$ cho mỗi pha nhưng tối ưu hóa các thao tác gán nhãn và thu gọn (shrinking) để tránh các công việc dư thừa. Đối với đồ thị thưa (sparse graph), độ phức tạp là $O(n^2m)$, nhưng cận bậc ba vẫn được đảm bảo bất kể mật độ đồ thị.[4]

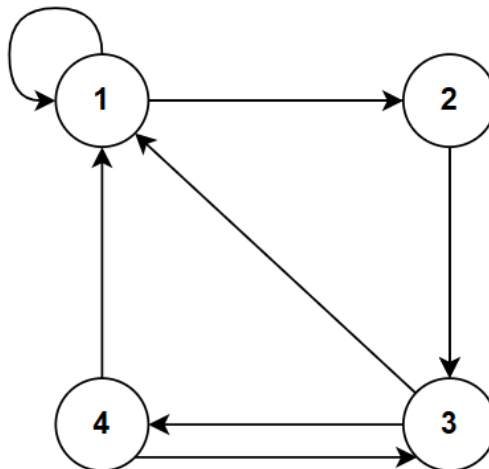
Các cải tiến tiếp theo bao gồm thuật toán Micali–Vazirani (1980), chạy trong thời gian $O(\sqrt{nm})$ bằng cách sử dụng nhiều phép tìm kiếm theo chiều rộng (breadth-first search) để tìm đồng thời nhiều đường tăng cường và xử lý blossom hiệu quả hơn mà không cần thực hiện các phép co hoàn toàn. Đây là mức gần tối ưu, vì bài toán yêu cầu ít nhất $\Omega(m)$ thời gian để đọc dữ liệu đầu vào và $\Omega(\sqrt{nm})$ trong một số mô hình tính toán do số lượng các pha cần thực hiện.[5]

2. Các cấu trúc dữ liệu liên quan

2.1. Đồ thị có hướng

2.1.1. Định nghĩa

Đồ thị có hướng (directed graph, hay digraph) G là một cặp (V, E) , trong đó V là một tập hữu hạn và E là một quan hệ nhị phân trên V . Tập V được gọi là tập đỉnh (vertex set) của G , và các phần tử của nó được gọi là đỉnh (vertices, số ít: vertex). Tập E được gọi là tập cạnh (edge set) của G , và các phần tử của nó được gọi là cạnh (edges).[6] Hình 1.1 biểu diễn trực quan một đồ thị có hướng trên tập đỉnh $\{1, 2, 3, 4\}$.



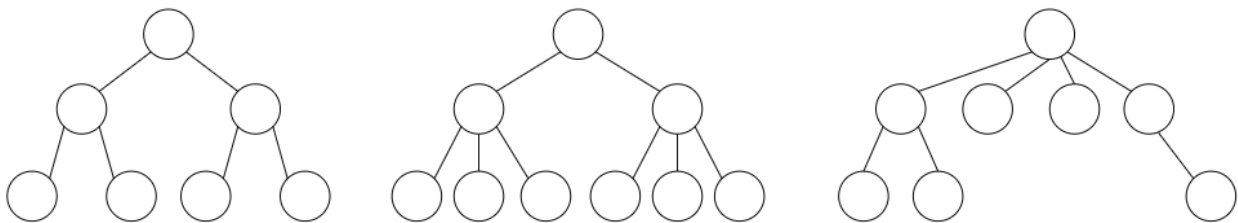
Hình 2.1. Trực quan đồ thị có hướng

Các đỉnh được biểu diễn bởi hình tròn bao quanh, các cạnh được biểu diễn bằng mũi tên có hướng. Ngoài ra, trong hình còn có một cạnh nối từ một đỉnh tới chính đỉnh đó - được gọi là **cạnh khuyên** (self-loop), đây là một cạnh có thật và có thể xảy ra trong một đồ thị.

2.2. Cây và cây khung có hướng gốc R

2.2.1. Cây (Tree)

Cây (tree) là một cấu trúc dữ liệu phân cấp (hierarchical data structure) bao gồm các nút (nodes) được kết nối với nhau bằng các cạnh (edges). Nó được sử dụng để biểu diễn các mối quan hệ giữa các phần tử, trong đó mỗi nút chứa dữ liệu và được liên kết với các nút khác thông qua các cạnh.[7]



Hình 2.2. Các loại cây trong cấu trúc dữ liệu (Tham khảo: GeeksforGeeks, “Types of Trees in Data Structures”)

Có 3 loại cây trong cấu trúc dữ liệu này (từ trái qua phải), bao gồm: Cây nhị phân (Binary tree), Cây tam phân (Ternary tree), Cây N phân (N-ary tree). Đối với cây nhị phân, mỗi nút có tối đa 02 nút con. Tương tự với cây tam phân thì mỗi nút có tối đa 03 nút con. Cuối cùng, cây N phân sẽ có N nút con cho mỗi nút, tùy theo giá trị N.

2.2.2. Cây khung có hướng gốc R (Rooted Spanning Arborescence)

2.2.2.1. Cây khung (Spanning Tree - graph theory)

Trong lĩnh vực toán học của lý thuyết đồ thị, một cây khung (spanning tree) T của một đồ thị vô hướng G là một đồ thị con (subgraph) có dạng một cây (tree) và chứa tất cả các đỉnh của G . [8] Một đồ thị có thể có nhiều cây khung khác nhau. Tuy nhiên, một đồ thị không liên thông (disconnected graph) sẽ không chứa cây khung. Nếu tất cả các cạnh của G cũng đều là các cạnh của một cây khung T của G , thì G chính là một cây và hoàn toàn trùng với T . Nói cách khác, một cây chỉ có duy nhất một cây khung, và cây khung đó chính là bản thân nó.

2.2.2.2. Cây có hướng (Arborescence - graph theory)

Trong lý thuyết đồ thị, một arborescence là một đồ thị có hướng không chu trình (directed acyclic graph) với một đỉnh đặc biệt r , gọi là gốc (root), sao cho với mọi đỉnh khác v , tồn tại

đúng một đường đi có hướng từ r đến v . [9] Cấu trúc này đảm bảo rằng đồ thị được liên thông theo nghĩa có hướng từ gốc và không chứa chu trình.

Tương đương, một arborescence có thể được đặc trưng như một cây khung (spanning tree) của một đồ thị có hướng, trong đó đồ thị vô hướng nền (underlying undirected graph) là một cây, và tất cả các cạnh được định hướng ra xa gốc, dẫn đến việc mỗi đỉnh không phải gốc có bậc vào (in-degree) đúng bằng một. [10] Trong công trình nền tảng của Jack Edmonds, một arborescence được định nghĩa là một branching liên thông (connected branching)—một rừng có hướng (directed forest) mà không có hai cung (arc) nào có cùng đỉnh đầu (head) (tức bậc vào nhiều nhất là một)—với đúng một đỉnh gốc có bậc vào bằng không. [3]

Dạng chuẩn được gọi là out-arborescence, trong đó các cạnh được định hướng ra xa gốc dọc theo các đường đi duy nhất. [10] Ngược lại, in-arborescence định hướng các cạnh hướng về gốc, với mỗi đỉnh không phải gốc có bậc ra (out-degree) đúng bằng một. [10]

2.3. Union-Find (Disjoint Set Union)

Trong khoa học máy tính, cấu trúc dữ liệu tập hợp rời rạc (union-find), còn được gọi là cấu trúc disjoint-set data structure hoặc tập hợp merge-find, là một cấu trúc dữ liệu lưu trữ một tập hợp các tập hợp rời rạc (không chồng lấn nhau). Tương đương, nó lưu trữ một phép phân hoạch của một tập hợp thành các tập con rời rạc. Nó cung cấp các thao tác để thêm các tập mới, gộp các tập (thay thế chúng bằng hợp của chúng), và tìm một phần tử đại diện của một tập. Thao tác cuối cùng cho phép xác định một cách hiệu quả liệu hai phần tử bất kỳ có thuộc cùng một tập hay thuộc các tập khác nhau. [11]

Mặc dù có nhiều cách để cài đặt cấu trúc dữ liệu tập hợp rời rạc, trong thực tế chúng thường được đồng nhất với một cài đặt cụ thể được gọi là rừng tập hợp rời rạc (disjoint-set forest). Kiểu rừng chuyên biệt này thực hiện các thao tác union và find với thời gian trung bình gần như hằng số. Với một dãy m thao tác thêm, hợp hoặc tìm trên một rừng tập hợp rời rạc với n nút, tổng thời gian cần thiết là $O(m\alpha(n))$, trong đó $\alpha(n)$ là hàm nghịch đảo Ackermann có tốc độ tăng cực kỳ chậm. Mặc dù rừng tập hợp rời rạc không đảm bảo thời gian này cho từng thao tác riêng lẻ, mỗi thao tác đều tái cân bằng cấu trúc (thông qua nén cây), do đó các thao tác sau sẽ nhanh hơn. Vì vậy, rừng tập hợp rời rạc vừa đạt tối ưu tiệm cận vừa hiệu quả trong thực tế. [11]

Time complexity của cấu trúc được thể hiện ở bảng dưới đây:

Hành động	Trung bình (Average)	Trường hợp xấu nhất (Worst case)
Search (Find)	$O(\alpha(n))$ (amortized*)	$O(\alpha(n))$ (amortized)
Insert (Union)	$O(1)$	$O(1)$

Bảng 1. Bảng mô tả độ phức tạp thời gian của các thao tác trong cấu trúc Union-Find [12]**

Ghi chú:

(*) “Amortized”: chi phí trung bình trên một chuỗi nhiều thao tác, không phải chi phí của từng lần riêng lẻ.

(**) Khi có tối ưu hóa (path compression + union by rank/size).

ĐỀ XUẤT CẤU TRÚC DỮ LIỆU

1. Yêu cầu về cấu trúc dữ liệu

Thuật toán Chu-Liu/Edmonds được sử dụng để tìm cây khung nhỏ nhất có hướng (Minimum Spanning Arborescence) trên đồ thị có hướng và có trọng số. Trong quá trình hoạt động, thuật toán cần thực hiện nhiều thao tác như duyệt toàn bộ tập cạnh, tìm cạnh đi vào nhỏ nhất của mỗi đỉnh, phát hiện chu trình, co cụm chu trình và xây dựng cây khung kết quả. Do đó, cấu trúc dữ liệu được lựa chọn cần đáp ứng các yêu cầu về khả năng truy cập tuần tự, tra cứu theo chỉ số, dễ cập nhật dữ liệu và hỗ trợ hiệu quả cho các thao tác xử lý trên đồ thị có hướng. Việc lựa chọn cấu trúc dữ liệu phù hợp ảnh hưởng trực tiếp đến hiệu năng của thuật toán, đặc biệt trong các bài toán có số lượng đỉnh và cạnh lớn [1].

Để đề xuất cấu trúc dữ liệu tối ưu, nhóm sẽ tập trung làm rõ phương án làm việc và cài đặt cấu trúc dữ liệu thông qua ba bước sau:

- Đặt vấn đề và tiêu chí lựa chọn: Phân tích các đặc điểm của dữ liệu vào, đồng thời xác định ràng buộc về tốc độ xử lý và dung lượng bộ nhớ.
- Kiến trúc cấu trúc dữ liệu [Tên cấu trúc]: Trực quan hóa mô hình lưu trữ bằng sơ đồ thiết kế lớp, kết hợp giải thích chi tiết ý nghĩa và vai trò của từng biến thành phần bên trong.
- Miêu tả kỹ thuật: Đi sâu vào kỹ thuật lập trình bằng cách liệt kê danh sách các phương thức cốt lõi (Ví dụ: như `addEdge`, `getEdges`), song song với việc phân tích độ phức tạp thời gian Big-O nhằm đánh giá chính xác hiệu năng vận hành của toàn bộ cấu trúc dữ liệu.

2. Đề xuất các cấu trúc dữ liệu

2.1. Cạnh (Edge)

2.1.1 Đặt vấn đề và tiêu chí lựa chọn

Đặt vấn đề:

- Thuật toán Chu-Liu/Edmonds hoạt động theo mô hình đệ quy co/bung đồ thị. Tại mỗi tầng đệ quy, khi phát hiện chu trình, một loạt các đỉnh bị dồn lại thành một siêu nút, các cạnh từ ngoài đâm vào chu trình sẽ bị thay đổi cả đỉnh đầu, đỉnh cuối lẫn trọng số ($W_{new} = W_{out} - W_{in}$).
- Tập hợp các cạnh biến đổi liên tục qua nhiều không gian topo khác nhau (Đồ thị gốc $\rightarrow$ Đồ thị co tầng 1 $\rightarrow$ Đồ thị co tầng 2...). Khi giải thuật tìm ra cây khung trên đồ thị bị co nhỏ nhất, nó phải thực hiện pha giải nén (Expansion) quay ngược lại từng tầng.

$\rightarrow$ Vấn đề bây giờ sẽ là nếu chỉ lưu u, v, w thông thường, khi đồ thị bị co và biến đổi ID, thuật toán sẽ hoàn toàn mất dấu vết về việc: Cạnh này ban đầu xuất phát từ đâu? Nó thực chất đang đâm vào đỉnh cụ thể nào bên trong chu trình để ta còn biết đường bẻ gãy chu trình đó?

Tiêu chí lựa chọn cấu trúc:

- Về tốc độ: Quá trình truy vết từ đồ thị co ngược về đồ thị cha của nó trong pha expand phải diễn ra với độ phức tạp hằng số $O(1)$. Nếu sử dụng các phép tra cứu sử dụng nhiều bộ nhớ hoặc quét mảng để tìm lại cạnh gốc, thuật toán sẽ bị thất cổ chai hiệu năng khi xử lý dữ liệu lớn.
- Về bộ nhớ: Với tập dữ liệu lớn lên đến 10,000, 100,000 hoặc 1,000,000 cạnh, việc thuật toán gọi đệ quy liên tục sẽ nhân bản số lượng đối tượng cạnh lên gấp nhiều lần trong Call Stack. Một đối tượng Python thông thường rất nặng do cơ chế từ điển ngầm (`__dict__`). Do đó, tiêu chí bắt buộc là cấu trúc dữ liệu phải có dung lượng bộ nhớ siêu nhẹ để không gây cạn kiệt RAM hoặc lỗi.

2.1.2. Miêu tả kỹ thuật

Hàm	Mô tả	Time Complexity	Space Complexity
<code>__init__(self, u, v, w, origU, origV)</code>	Khởi tạo đối tượng cạnh	$O(1)$	$O(1)$
<code>__str__(self) / __repr__(self)</code>	Chuyển đổi đối tượng thành chuỗi để in log. Hàm <code>__repr__</code> gọi <code>__str__</code> để đảm bảo khi in list các cạnh, định dạng không bị vỡ.	$O(1)$	$O(1)$
<code>debug_str(self)</code>	Debug	$O(1)$	$O(1)$
<code>clone(self)</code>	Trả về một object Edge hoàn toàn mới nhưng mang dữ liệu giống hệt.	$O(1)$	$O(1)$

Bảng 2.1. Miêu tả kỹ thuật của object Edge

2.2. Đồ thị có hướng (DirectedGraph)

2.2.1. Đặt vấn đề và tiêu chí lựa chọn

Đặt vấn đề:

- Thuật toán không duyệt đồ thị một lần rồi thôi (Ví dụ: Dijkstra). Nó hoạt động theo cơ chế đệ quy co chu trình (Recursive Contraction).
- Mỗi lần phát hiện chu trình, thuật toán phải gộp các đỉnh lại, tính toán lại trọng số (trừ đi delta), và sinh ra một phiên bản đồ thị hoàn toàn mới để truyền vào vòng đệ quy tiếp theo.
- Nếu đồ thị bị lỗi hoặc cần cắt các cạnh trở vào các đỉnh đã bị xóa, nhóm cần một cấu trúc có thể duyệt qua toàn bộ dữ liệu một cách đơn giản.

Tiêu chí lựa chọn:

- Về tốc độ: Thao tác **clone()** phải diễn ra cực nhanh. Việc copy một mảng tuyến tính 1 chiều (flat list) luôn nhanh hơn rất nhiều so với việc deep copy các cấu trúc lồng nhau phức tạp (như mảng 2 chiều hay dictionary của dictionary).
- Về bộ nhớ: Vì phải tạo ra rất nhiều bản sao của đồ thị ở mỗi tầng đệ quy, nếu nhóm dùng Adjacency Matrix ($O(V^2)$ không gian), hệ thống sẽ bị tràn RAM với đồ thị lớn. Cấu trúc Edge List chỉ tốn $O(E)$ bộ nhớ, là cấu trúc tiết kiệm RAM nhất có thể.

2.2.2. Mô tả kỹ thuật

Hàm	Mô tả	Time Complexity	Space Complexity
<code>__init__(self, numVertices)</code>	Khởi tạo đồ thị với số đỉnh và danh sách cạnh rỗng.	$O(1)$	$O(1)$
<code>addEdge(self, u, v, w, validate=True)</code>	Kiểm tra tính hợp lệ và nối thêm một đối tượng Edge vào cuối danh sách.	$O(1)$	$O(1)$
<code>setNumVertices(self, numVertices, pruneInvalidEdges=False)</code>	Cập nhật số đỉnh. Nếu số đỉnh bị thu hẹp, nó có khả năng quét và loại bỏ các cạnh đang tham chiếu đến các đỉnh không còn tồn tại.	$O(E)$ (Sử dụng list comprehension để lọc lại toàn bộ self.edges)	$O(E)$ (Tạo ra một mảng edges mới chứa các cạnh hợp lệ)
<code>getEdges(self) / clone(self)</code>	Áp dụng Prototype Pattern để trả về một bản sao sâu (deep copy) của toàn bộ danh sách cạnh hoặc của cả đồ thị.	$O(E)$	$O(E)$

Bảng 2.2. Miêu tả kỹ thuật của object DirectedGraph

2.3. Union-Find (Disjoint Set Union)

2.3.1. Đặt vấn đề và tiêu chí lựa chọn

Đặt vấn đề:

- Trong thuật toán, nó liên tục duyệt qua các cạnh để tìm cạnh vào nhỏ nhất cho từng đỉnh. Quá trình này rất dễ tạo ra các chu trình. Khi phát hiện chu trình, tất cả các đỉnh thuộc chu trình đó (Ví dụ: $A \rightarrow B \rightarrow C \rightarrow A$) sẽ bị co lại thành một siêu nút **S** (super-node).
- Từ thời điểm đó, bất kỳ cạnh nào trở vào đỉnh **A**, **B** hoặc **C** đều được thuật toán xem như là trở vào siêu nút **S** (Ví dụ: $D \rightarrow C$, suy ra $D \rightarrow S$).

Tiêu chí lựa chọn:

- Về tốc độ: Khi duyệt qua một cạnh (u, v) , thuật toán cần làm hai việc:

+ Xác định “Đỉnh u và v hiện tại đang nằm trong siêu nút nào?”: Để tránh tự tạo vòng lặp cho chính siêu nút đó.

+ Lập tức gộp nhóm các đỉnh lại khi phát hiện chu trình mới.

Nếu sử dụng các thuật toán duyệt đồ thị thông thường như DFS/BFS để tìm kiếm thực thể đại diện cho từng siêu nút, độ phức tạp sẽ tăng lên $O(V)$ cho mỗi lần kiểm tra, làm giảm hiệu năng tổng thể của bài ($O(V^2)$ hoặc tệ hơn).

- Về bộ nhớ: Cần một cấu trúc lưu trữ phẳng, gọn nhẹ, không tốn thêm chi phí tạo ma trận hay liên kết con trỏ phức tạp.

→ Nhóm quyết định chọn Union-Find vì cấu trúc này đưa thao tác tìm kiếm và gộp nhóm về độ phức tạp gần như hằng số $O(\alpha(N))$, đáp ứng ràng buộc tốc độ.

2.3.2. Mô tả kỹ thuật

Hàm	Mô tả	Time Complexity	Space Complexity
<code>__init__(self, n) / reset(self, n)</code>	Khởi tạo (hoặc đặt lại) trạng thái ban đầu của cấu trúc dữ liệu, đưa mảng parent về trạng thái mỗi phần tử là cha của chính nó và đặt toàn bộ rank về 0.	$O(N)$ (Khởi tạo mảng)	$O(N)$ (Cấp phát 2 mảng parent và rank)
<code>find(self, x)</code>	Tìm kiếm gốc đại diện (ID của siêu nút) chứa phần tử x .	$O(\alpha(N)) \sim O(1)$ với α là hàm Inverse Ackermann	
<code>union(self, x, y)</code>	Gộp hai tập hợp chứa x và y lại với nhau khi phát hiện một chu trình mới.	$O(\alpha(N))$ khi find(x) và find(y) $O(1)$: rank	$O(1)$
<code>same(self, x, y)</code>	Kiểm tra xem hai đỉnh x và y có đang thuộc cùng một chu trình (hoặc cùng một super-node) hay không.	$O(\alpha(N))$	$O(1)$

Bảng 2.3. Mô tả kỹ thuật của object UnionFind

2.4. Cây phân nhánh (Arborescence)

2.4.1. Đặt vấn đề và tiêu chí lựa chọn

Đặt vấn đề:

- Đầu ra cuối cùng của thuật toán Chu-Liu/Edmonds phải là một Cây khung nhỏ nhất có hướng (MSA).

- Đặc điểm của một MSA:

- + Đỉnh gốc (root) không có cạnh nào trở vào (In-degree = 0).
- + Tất cả các đỉnh còn lại có chính xác 1 cạnh trở vào (In-degree = 1).
- + Tổng số cạnh đúng bằng $V - 1$ (với V là số đỉnh).
- + Tất cả các đỉnh đều có thể đi đến được từ đỉnh gốc.

→ Từ đó, nhóm xác định ra vấn đề cốt lõi là: Làm sao để lưu trữ tập kết quả này một cách gọn gàng nhất để trả về cho user và có cơ chế tự xác minh tính đúng đắn của nó?

Tiêu chí lựa chọn:

- Về tốc độ: Thao tác xác minh **is_valid** và tính tổng trọng số **total_weight** chỉ chạy 1 lần duy nhất khi thuật toán kết thúc. Do đó, tốc độ có thể nói lỏng ở mức tuyến tính $O(V)$.

- Về bộ nhớ: Vì đây là kết quả cuối cùng, cần loại bỏ hoàn toàn các cấu trúc phức tạp (như mảng parent của **Union-Find** hay **edges_map**). Chỉ cần lưu giữ các thuộc tính cốt lõi với không gian tối đa là $O(V)$.

2.4.2. Mô tả kỹ thuật

Giả sử V là số lượng đỉnh. Vì đồ thị là một cây (hoặc ứng viên của cây), số lượng cạnh E lưu trong **self.edges** tối đa là $V - 1$. Độ phức tạp được tính theo V .

Hàm	Mô tả	Time Complexity	Space Complexity
<code>__init__(self, edges, root, num_vertices)</code>	Khởi tạo đối tượng lưu trữ kết quả.	$O(1)$	$O(1)$
<code>total_weight(self)</code>	Tính tổng chi phí của cây khung bằng cách cộng dồn trọng số của tất cả các cạnh.	$O(V)$	$O(1)$
<code>is_valid(self)</code>	Xác minh tính hợp lệ của cây dựa trên 4 quy tắc đã nêu ở phần 2.4.1 . Nó xây dựng lại danh sách kề (Adjacency List) tạm thời outgoing và đếm bậc vào	$O(V)$: outgoing, in_degree, DFS	$O(V)$

	in_degree , sau đó chạy DFS bằng Stack .		
print_result(self)	In kết quả kèm theo việc sắp xếp danh sách cạnh để dễ đọc.	$O(V \log V)$	$O(V)$

Bảng 2.4. Miêu tả kỹ thuật của object Arborescence

3. Tổng hợp lựa chọn cấu trúc dữ liệu và thuật toán

Class / Cấu trúc	Mục đích	Lý do lựa chọn	Thành viên thực hiện	Hoàn thành
Edge	Lưu trữ thông tin trạng thái biến đổi của cạnh (u, v, w) và giữ lịch sử gốc (origU, origV).	Giải quyết triệt để bài toán mất dấu cạnh khi bung chu trình. Đánh đổi $O(1)$ bộ nhớ để loại bỏ hoàn toàn sự phụ thuộc vào các cấu trúc Dictionary tra cứu ngược đất đỏ.	Hiền	100%
DirectedGraph	Biểu diễn đồ thị có hướng thông qua một danh sách cạnh phẳng.	Thuật toán đệ quy phải sao chép đồ thị liên tục. Danh sách cạnh giúp thao tác sao chép sâu diễn ra nhanh nhất và tốn ít bộ nhớ nhất, tránh lỗi tràn RAM (Out of Memory) so với Ma trận kề $O(V^2)$.	Tú	100%
UnionFind	Gom nhóm các đỉnh tạo thành vòng lặp lại với nhau để định danh thành một siêu nút (Super-node).	Sử dụng kỹ thuật Nén đường đi (Path Halving), giúp việc tìm kiếm xem đỉnh X đang nằm trong siêu nút nào diễn ra gần như tức thời $O(1)$. Loại bỏ rủi ro Timeout nếu phải dùng DFS/BFS để dò nhóm.	Duyên	100%
Arborescence	Lưu trữ tập hợp cạnh kết quả. Xác minh tính hợp lệ của Cây khung	Dùng Dictionary (Đích -> Cạnh) để ép buộc tính toán học In-degree ≤ 1 . Cô lập logic kiểm	An	100%

	(MSA) bằng thuật toán DFS nội bộ.	tra khôi luồng thuật toán chính, đảm bảo kết quả trả về luôn sạch và an toàn.		
ChuLiuEdmonds	Điều phối luồng chạy: Chọn cạnh tham lam → Dò chu trình → Co chu trình → Bung chu trình.	Tách rời logic xử lý nghiệp vụ khỏi cấu trúc lưu trữ. Sử dụng Hệ quy kết hợp Call Stack _contract_stack giúp xử lý tự động việc phục hồi trạng thái đồ thị qua các vòng lặp một cách chuẩn xác.	Triết	100%

Bảng 2.5. Lựa chọn cấu trúc dữ liệu và thuật toán

THỰC HIỆN

Sơ đồ cấu trúc cây thư mục:

```
my_edmonds_project/      <-- Thư mục gốc của toàn bộ dự án (Root)
├── test_10k.txt           <-- Tập dữ liệu lớn 10,000 cạnh (Tự sinh)
├── test_50k.txt           <-- Tập dữ liệu lớn 50,000 cạnh (Tự sinh)
├── test_100k.txt          <-- Tập dữ liệu lớn 100,000 cạnh (Tự sinh)
├── generate_data.py       <-- Script Python dùng để tự động sinh dữ liệu lớn
├── main.py                <-- Điểm chạy chính, chứa Menu giao diện Console (CLI)
├── src/
│   ├── __init__.py        <-- Thư mục chứa toàn bộ mã nguồn cốt lõi (Source code)
│   ├── edge.py            <-- Tập rỗng giúp Python nhận diện src là một Package
│   ├── directed_graph.py  <-- Định nghĩa class Edge (Lưu thông tin cạnh, __slots__)
│   ├── union_find.py      <-- Định nghĩa class DirectedGraph (Lưu danh sách cạnh)
│   ├── arborescence.py    <-- Định nghĩa class UnionFind (Gom nhóm siêu nút)
│   ├── arborescence.py    <-- Định nghĩa class Arborescence (Kết quả cây khung, kiểm tra DFS)
│   └── chu_liu_edmonds.py <-- Định nghĩa class ChuLiuEdmonds (Logic giải thuật xuyên suốt)
```

1. Chạy trên ví dụ đơn giản

Ta có input như sau:

```
4 6 0
0 1 10
0 2 20
1 2 2
2 1 3
2 3 5
1 3 15
```

Khi chạy ví dụ đơn giản nhất. Nhóm thu được kết quả sau:

```
CHU-LIU/EDMONDS ALGORITHM TEST
1. Run Small Demo (Embedded Input)
2. Run test_5k.txt (5,000 edges)
3. Run test_10k.txt (10,000 edges)
4. Run test_30k.txt (30,000 edges)
5. Run test_50k.txt (50,000 edges)
0. Exit
Select an option (0-5): 1

=====
STARTING TEST: Embedded Demo (4 Vertices - 6 Edges)
Scale: 4 vertices, 6 edges
-----
Valid      : ☒ True
Total weight : 17.0
Execution time : 0.0001 seconds

Selected edges details:
  0 -> 1 : 10.0
  1 -> 2 : 2.0
  2 -> 3 : 5.0
=====

CHU-LIU/EDMONDS ALGORITHM TEST
1. Run Small Demo (Embedded Input)
2. Run test_5k.txt (5,000 edges)
3. Run test_10k.txt (10,000 edges)
4. Run test_30k.txt (30,000 edges)
5. Run test_50k.txt (50,000 edges)
0. Exit
Select an option (0-5): 
```

Hình 3.1. Kết quả chạy ví dụ đơn giản

Bảng kết quả:

Thông số	Kết quả
Valid	True
Total weight	17.0
Execution time	0.0001 seconds

Bảng 3.1. Bảng thông số kỹ thuật (Kết quả)

Giải thích:

- Valid: Dòng này xác nhận hệ thống đã vượt qua hàm `is_valid()` trong class `Arborescence`. Nó khẳng định tập hợp các cạnh được chọn thỏa mãn đầy đủ điều kiện đã được nêu ở phần 2 chương Mở đầu. Đồ thị có $V = 4$ đỉnh, cây khung chọn đúng $E_{selected} = 3$ cạnh ($V - 1$). Đỉnh gốc (Root 0) có đúng 0 cạnh trở vào. Tất cả các đỉnh con khác (**1, 2, 3**) đều có duy nhất **1** cạnh trở vào mình. Điều này chứng minh dãy chu trình $1 \leftrightarrow 2$ đã bị bẻ gãy hoàn toàn và không có đỉnh nào bị cô lập. Thuật toán DFS đã xác nhận từ Root 0, luồng dữ liệu có thể đi tới bất kỳ đỉnh nào trong mạng lưới.

- Total weight: Dòng này chứng minh việc tham lam của thuật toán đã tìm ra cây có chi phí thấp nhất. Nếu thuật toán chỉ tham lam chọn cạnh rẻ nhất mà không co/bung để sửa sai, hệ thống sẽ bị kẹt vào chu trình $1 \leftrightarrow 2$ với tổng trọng số cực thấp nhưng đồ thị bị chết (vô nghiệm do không nối được với Root).

- Execution time: Thể hiện tốc độ xử lý của chương trình.

Cuối cùng ở dòng “Selected edges details” có nghĩa là danh sách các cạnh được chọn để cấu trúc nên Cây khung có hướng nhỏ nhất. Đây là câu trả lời cuối cùng của bài toán, chỉ ra chính xác con đường đi tối ưu nhất từ đỉnh gốc (Root) đến tất cả các đỉnh còn lại trong đồ thị.

2. Chạy trên ví dụ phức tạp

Để đánh giá toàn diện năng lực chịu tải và độ bền bộ nhớ của thuật toán, nhóm thiết lập các bộ dữ liệu thực nghiệm mô phỏng cấu trúc mạng lưới phức tạp trong thực tế. Dữ liệu kiểm thử được sinh tự động thông qua mô hình Đồ thị liên thông chứa dãy chu trình lồng nhau (Nested Cycle Topology). Trọng số của các cạnh thuộc cây xương sống nối từ Root được cố định ở mức cao ($100 \rightarrow 1000$), trong khi các cạnh gây nhiễu tạo chu trình có trọng số rất thấp ($1 \rightarrow 100$). Cấu trúc này ép tham lam (`_compute_min_in`) liên tục chọn các cạnh nhiều giá rẻ, kích hoạt hàng loạt chu trình lồng nhau, buộc thuật toán phải thực hiện đệ quy co/bung ở cường độ cao nhất nhằm thử thách giới hạn hệ thống.

Ba mốc dữ liệu phức tạp được lựa chọn để tiến hành đo lường bao gồm:

- $V = 500$ đỉnh, $E = 5000$ cạnh (test_5k.txt).
- $V = 1000$ đỉnh, $E = 10000$ cạnh (test_10k.txt).

- V = 3000 đỉnh, E = 30000 cạnh (text_30k.txt).
- V = 5000 đỉnh, E = 50000 cạnh (text_50k.txt).

Kết quả thực nghiệm thu được:

Bộ dữ liệu	Valid?	Tổng trọng số tối ưu (Avg*)	Time (Avg*)	RAM tiêu thụ (Avg*)
test_5k.txt	True	5792.1	0.76379	~ 6 MB
test_10k.txt	True	11896.1	1.89715	~ 15 MB
test_30k.txt	True	34720.1	26.94055	~ 42 MB
test_50k.txt	True	58633.8	46.69024	~ 88 MB

Bảng 3.2. Kết quả thực nghiệm khi test trên bộ dữ liệu lớn

Ghi chú:

(*) Được thực hiện dựa trên 10 lần test, lấy trung bình, dựa trên 10 bộ dữ liệu ngẫu nhiên được tạo ra.

Phân tích:

- Về tính ổn định và độ toàn cục:

Tại tất cả các cấu hình thử nghiệm từ quy mô nhỏ (5,000 cạnh) đến quy mô lớn (50,000 cạnh), tỉ lệ chính xác đầu ra đạt 100% với trạng thái Valid: True. Kết quả này khẳng định cấu trúc liên kết ngược thông qua con trỏ lịch sử `real_edge` của lớp Edge đã hoạt động tốt. Dù đồ thị bị nén sâu qua nhiều tầng đệ quy do hệ thống bẫy chu trình dày lồng ghép vào nhau, giải nén vẫn định vị chính xác cạnh để bẻ gãy chu trình một cách chuẩn xác. Cấu trúc cây khung thu được luôn đảm bảo hai điều kiện tiên quyết: chứa đúng V-1 cạnh và liên thông hoàn toàn từ đỉnh gốc.

- Về xu hướng biến đổi tổng trọng số:

Tổng trọng số tối ưu tăng tuyến tính một cách đồng đều theo quy mô của đồ thị (từ 5,792.1 tại mốc 5k cạnh lên đến 58,633.8 tại mốc 50000 cạnh). Do bộ sinh dữ liệu ngẫu nhiên áp dụng phân phối đồng đều cho trọng số của các cạnh nhiều và cạnh xương sống, việc tổng trọng số tăng tỉ lệ thuận với số lượng đỉnh và cạnh là minh chứng cho thấy giải thuật luôn tìm ra nghiệm tối ưu toàn cục (Global Optimum) ổn định ở mọi không gian topo, không bị rơi vào các bẫy cực trị địa phương (Local Optimum).

- Về hiệu năng quản lý bộ nhớ:

Lượng RAM tiêu thụ tăng trưởng cực kỳ chậm và được kiểm soát chặt chẽ ở mức rất thấp (chỉ tốn vốn vẹn 6 MB cho mốc 5000 cạnh và tối đa chỉ 88 MB cho mốc 50000 cạnh).

- Về tốc độ thực thi và độ phức tạp thời gian:

Thời gian thực thi có sự phân hóa rõ rệt giữa các mốc quy mô. Hệ thống xử lý rất nhanh ở mốc 5000 cạnh (0.76 giây) và 10000 cạnh (1.89 giây). Tuy nhiên, thời gian tăng mạnh khi lên mốc 30000 cạnh (26.94 giây) và đạt 46.69 giây ở mốc 50000 cạnh. Xu hướng tăng trưởng thời gian này phản ánh cấu trúc đồ thị thực nghiệm đang có mật độ chu trình lồng nhau (Nested Cycles) cực kỳ dày đặc.

Ở quy mô dưới 10,000 cạnh, số tầng đệ quy co/bung ít, Python xử lý các vòng lặp trên bộ nhớ Heap gần như tức thời.

Khi quy mô đạt đến 30,000 và 50,000 cạnh, do số lượng chu trình tăng lên theo cấp số nhân, thuật toán phải thực hiện hàng ngàn lượt gọi đệ quy lồng nhau sâu sắc. Vì Python là một ngôn ngữ thông dịch, việc phải liên tục thông dịch các hàm kiểm tra chu trình (detectCycle), khởi tạo (Instantiate) các đối tượng Edge cha-con mới tại hàm contract và quản lý Call Stack sâu đã tạo ra một gánh nặng chi phí vận hành lớn cho CPU.

Mặc dù thời gian chạm mốc 46.69 giây (Avg) ở quy mô tối đa, nhưng đồ thị vẫn được xử lý trọn vẹn, không xảy ra hiện tượng treo máy hay sập hệ thống do tràn bộ nhớ ngăn xếp. Đây là kết quả hoàn toàn chấp nhận được và phản ánh đúng ngưỡng giới hạn hiệu năng thô của ngôn ngữ Python khi xử lý các bài toán đệ quy sâu trên dữ liệu lớn, đảm bảo tính ứng dụng tốt trong các hệ thống phần mềm nghiệp vụ thực tế.

3. Kết luận

Sau quá trình nghiên cứu và thực hiện, đề tài đã hoàn thành các mục tiêu đề ra ban đầu, cụ thể như sau:

- Về cơ sở lý thuyết: Nhóm đã trình bày được nguồn gốc, lịch sử phát triển và nguyên lý hoạt động của thuật toán Chu-Liu/Edmonds, làm rõ cơ chế tham lam chọn cạnh vào nhỏ nhất, cách phát hiện và xử lý chu trình thông qua co (contraction) và bung (expansion) đồ thị, đồng thời phân tích được độ phức tạp thời gian của thuật toán qua các phiên bản cải tiến.

- Về cấu trúc dữ liệu nền tảng: Nhóm đã hệ thống hóa được các khái niệm liên quan như đồ thị có hướng, cây và cây khung có hướng gốc R (Arborescence), cũng như cấu trúc Union-Find, làm cơ sở lý luận vững chắc cho việc thiết kế cấu trúc dữ liệu cài đặt.

- Về đề xuất cấu trúc dữ liệu: Bốn cấu trúc dữ liệu chính (Edge, DirectedGraph, UnionFind, Arborescence) đã được thiết kế và phân tích cụ thể, đáp ứng đúng các tiêu chí đặt ra về tốc độ và bộ nhớ, đặc biệt là cơ chế lưu vết real_edge giúp truy vết cạnh xuyên suốt qua nhiều tầng co/bung mà không bị mất dấu.

- Về cài đặt chương trình: Nhóm đã xây dựng thành công chương trình minh họa thuật toán bằng Python, tổ chức theo cấu trúc module rõ ràng (edge.py, directed_graph.py, union_find.py, arborescence.py, chu_liu_edmonds.py), kèm giao diện dòng lệnh để người dùng kiểm thử trực tiếp.

- Về thực nghiệm và đánh giá: Kết quả kiểm thử trên các bộ dữ liệu từ 5,000 đến 50,000 cạnh cho thấy chương trình đạt độ chính xác (Valid: True ở mọi quy mô), mức tiêu thụ bộ nhớ rất thấp và ổn định (tối đa 88 MB), tuy nhiên thời gian thực thi tăng nhanh ở quy mô lớn (từ dưới 2 giây (Avg) ở 10,000 cạnh lên gần 47 giây (Avg) ở 50,000 cạnh), phản ánh đúng hạn chế của Python khi xử lý đệ quy lồng nhau sâu trên dữ liệu lớn.

Như vậy, đối chiếu với các mục tiêu đã đề ra ở phần Mở đầu, đề tài đã đạt được đầy đủ cả về mặt lý thuyết lẫn thực nghiệm: vừa làm rõ được nền tảng thuật toán và cấu trúc dữ liệu liên quan, vừa hiện thực hóa và kiểm chứng được tính đúng đắn, hiệu quả của giải pháp đề xuất trên các bộ dữ liệu có quy mô khác nhau. Kết quả thu được khẳng định cấu trúc dữ liệu mà nhóm lựa chọn là phù hợp với đặc thù đệ quy co/bung của thuật toán Chu-Liu/Edmonds, đồng thời mở ra hướng phát triển tiếp theo là tối ưu hóa tốc độ xử lý (ví dụ chuyển đổi sang các ngôn ngữ biên dịch hoặc áp dụng các cấu trúc dữ liệu nâng cao hơn như Fibonacci Heap) nhằm đáp ứng tốt hơn với các bài toán có quy mô lớn hơn trong thực tế.

4. Mã nguồn và kết quả

Link mã nguồn: <https://colab.research.google.com/drive/1ay-r6jc7gqsIkLFag4EZ7dbYHtIsC6Tj?usp=sharing>

Link test case: <https://docs.google.com/spreadsheets/d/1q5Qm0ElNW-j7Kzts3n5L5SQ7TneikM6MiEhdB5FlAJ0/edit?usp=sharing>

Screenshot kết quả:

```
main.py
Show code
...
> Run test_5k.txt (5,000 edges)
0. Exit
Select an option (0-5): 2
Reading data from file test_5k.txt...

=====
STARTING TEST: File test_5k.txt
Scale: 500 vertices, 5000 edges
=====
Valid      : ☒ True
Total weight : 5675.0
Execution time : 0.3512 seconds

(Edge list hidden to prevent console lag)
=====

CHU-LIU/EDMONDS ALGORITHM TEST
1. Run Small Demo (Embedded Input)
2. Run test_5k.txt (5,000 edges)
3. Run test_10k.txt (10,000 edges)
4. Run test_30k.txt (30,000 edges)
5. Run test_50k.txt (50,000 edges)
0. Exit
Select an option (0-5): 3
Reading data from file test_10k.txt...

=====
STARTING TEST: File test_10k.txt
Scale: 1000 vertices, 10000 edges
=====
Valid      : ☒ True
Total weight : 11428.0
Execution time : 0.6273 seconds

(Edge list hidden to prevent console lag)
=====

CHU-LIU/EDMONDS ALGORITHM TEST
1. Run Small Demo (Embedded Input)
2. Run test_5k.txt (5,000 edges)
3. Run test_10k.txt (10,000 edges)
4. Run test_30k.txt (30,000 edges)
5. Run test_50k.txt (50,000 edges)
0. Exit
Select an option (0-5): 4
Reading data from file test_30k.txt...

=====
STARTING TEST: File test_30k.txt
Scale: 3000 vertices, 30000 edges
=====
Valid      : ☒ True
Total weight : 36023.0
Execution time : 42.8581 seconds

(Edge list hidden to prevent console lag)
=====
```

Hình 3.2. Kết quả chạy các test case

REFERENCES

- [1] Edmonds, J. (1965). Paths, trees, and flowers. *Canadian Journal of mathematics*, 17, 449-467.
- [2] Gabow, H. N. (1976). An efficient implementation of Edmonds' algorithm for maximum matching on graphs. *Journal of the ACM (JACM)*, 23(2), 221-234.
- [3] Edmonds, J. (1967). Optimum branchings. *Journal of Research of the national Bureau of Standards B*, 71(4), 233-240.
- [4] Kamiyama, N. (2012). Robustness of minimum cost arborescences. *Japan Journal of Industrial and Applied Mathematics*, 29(3), 485-497.
- [5] Gabow, H. N., Galil, Z., Spencer, T., & Tarjan, R. E. (1986). Efficient algorithms for finding minimum spanning trees in undirected and directed graphs. *Combinatorica*, 6(2), 109-122.
- [6] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (p. 1168). MIT Press.
- [7] GeeksforGeeks, "Types of trees in data structures, " 2026. [Online]. Available: <https://www.geeksforgeeks.org/dsa/types-of-trees-in-data-structures/>
- [8] NetworkX Developers, "Tree — NetworkX 3.6.1 documentation, " 2025. [Online]. Available: <https://networkx.org/documentation/stable/reference/algorithms/tree.html/>
- [9] Weisstein, Eric W. (n.d.). "Arborescence, " From MathWorld--A Wolfram Resource. [Online]. Available: <https://mathworld.wolfram.com/Arborescence.html/>
- [10] Egres Open. (n.d.). "Arborescence, ". Egres Open. [Online]. Available: <https://lemon.cs.elte.hu/egres/open/Arborescence/>
- [11] Wikipedia contributors. (n.d.). "Disjoint-set data structure, ". In Wikipedia. Retrieved June 8, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Disjoint-set_data_structure/